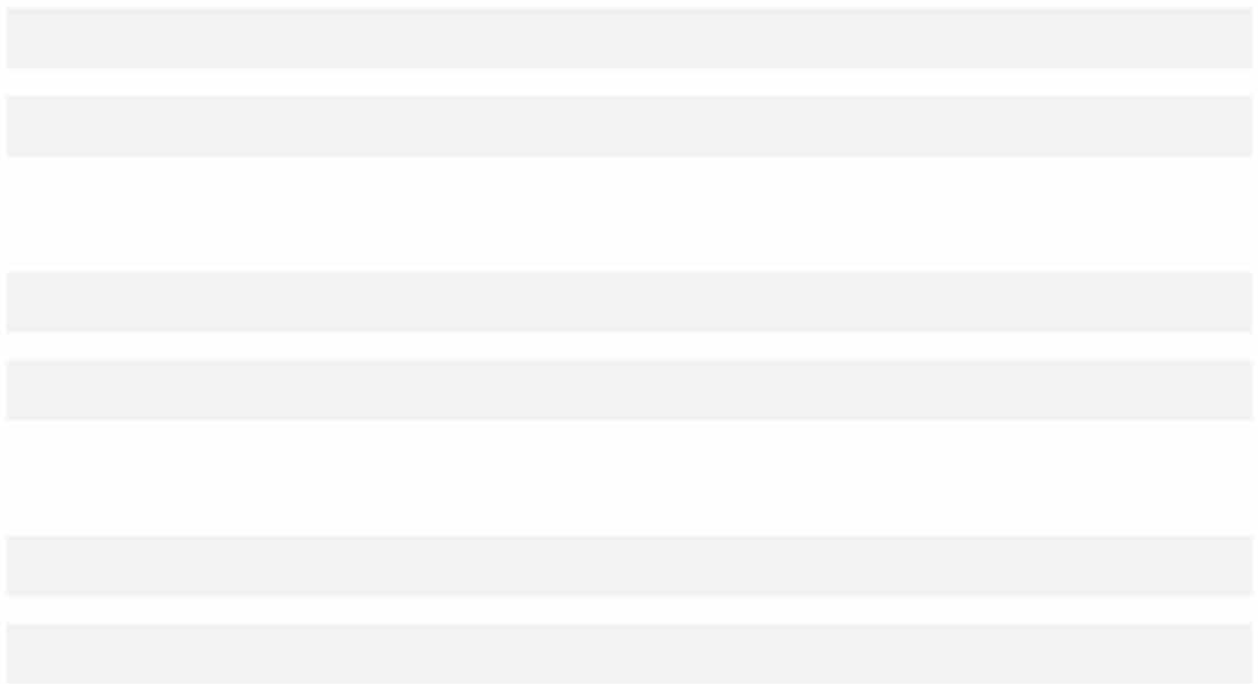




\* \*

## 14.7 Bidirectional Encoder Representations from Transformers (BERT)

We have introduced several word embedding models for natural language understanding. After pretraining, the output can be thought of as a matrix where each row is a vector that represents a word of a predefined vocabulary. In fact, these word embedding models are all *context-independent*. Let us begin by illustrating this property.

### 14.7.1 From Context-Independent to Context-Sensitive

Recall the experiments in [Section 14.4](#) and [Section 14.6](#). For instance, word2vec and GloVe both assign the same pretrained vector to the same word regardless of the context of the word (if any). Formally, a context-independent representation of any token  $x$  is a function  $f(x)$  that only takes  $x$  as its input. Given the abundance of polysemy and complex semantics in natural languages, context-independent representations have obvious limitations. For instance, the word “crane” in contexts “a crane is flying” and “a crane driver came” has completely different meanings; thus, the same word may be assigned different representations depending on contexts.

This motivates the development of *context-sensitive* word representations, where representations of words depend on their contexts. Hence, a context-sensitive representation of token  $x$  is a function  $f(x, c(x))$  depending on both  $x$  and its context  $c(x)$ . Popular context-sensitive representations include TagLM (language-model-augmented sequence tagger) ([Peters et al., 2017b](#)), CoVe (Context Vectors) ([McCann et al., 2017](#)), and ELMo (Embeddings from Language Models) ([Peters et al., 2018](#)).

For example, by taking the entire sequence as the input, ELMo is a function that assigns a representation to each word from the input sequence. Specifically, ELMo combines all the intermediate layer representations from pretrained bidirectional LSTM as the output representation. Then the ELMo representation will be added to a downstream task’s existing supervised model as additional features, such as by concatenating ELMo representation and the original representation (e.g., GloVe) of tokens in the existing model. On one hand, all the weights in the pretrained bidirectional LSTM model are frozen after ELMo representations are added. On the other hand, the existing supervised model is specifically customized for a given task. Leveraging different best models for different tasks at that time, adding ELMo improved the state of the art across six natural language processing tasks: sentiment analysis, natural language inference, semantic role labeling, coreference resolution, named entity recognition, and question answering.

### 14.7.2 From Task-Specific to Task-Agnostic

Although ELMo has significantly improved solutions to a diverse set of natural language processing tasks, each solution still hinges on a *task-specific* architecture. However, it is practically non-trivial to craft a specific architecture for every natural language processing task. The GPT (Generative Pre-Training) model represents an effort in designing a general *task-agnostic* model for context-sensitive representations ([Radford et al., 2018](#)). Built on a transformer decoder, GPT pre-trains a language model that will be used to represent text sequences. When applying GPT to a downstream task, the output of the language model will be fed into an added linear output layer to predict the label of the task. In sharp contrast to ELMo that freezes parameters of the pretrained model, GPT fine-tunes *all* the parameters in the pretrained transformer decoder during supervised learning of the downstream task. GPT was evaluated on twelve tasks of natural language inference, question answering, sentence similarity, and classification, and improved the state of the art in nine of them with minimal changes to the model architecture.

However, due to the autoregressive nature of language models, GPT only looks forward (left-to-right). In contexts “i went to the bank to deposit cash” and “i went to the bank to sit down”, as “bank” is sensitive to the context to its left, GPT will return the same representation for “bank”, though it has different meanings.

### 14.7.3 BERT: Combining the Best of Both Worlds

As we have seen, ELMo encodes context bidirectionally but uses task-specific architectures; while GPT is task-agnostic but encodes context left-to-right. Combining the best of both worlds, BERT (Bidirectional Encoder Representations from Transformers) encodes context bidirectionally and requires minimal architecture changes for a wide range of natural language processing tasks (Devlin et al., 2018). Using a pretrained transformer encoder, BERT is able to represent any token based on its bidirectional context. During supervised learning of downstream tasks, BERT is similar to GPT in two aspects. First, BERT representations will be fed into an added output layer, with minimal changes to the model architecture depending on nature of tasks, such as predicting for every token vs. predicting for the entire sequence. Second, all the parameters of the pretrained transformer encoder are fine-tuned, while the additional output layer will be trained from scratch. Fig. 14.7.1 depicts the differences among ELMo, GPT, and BERT.

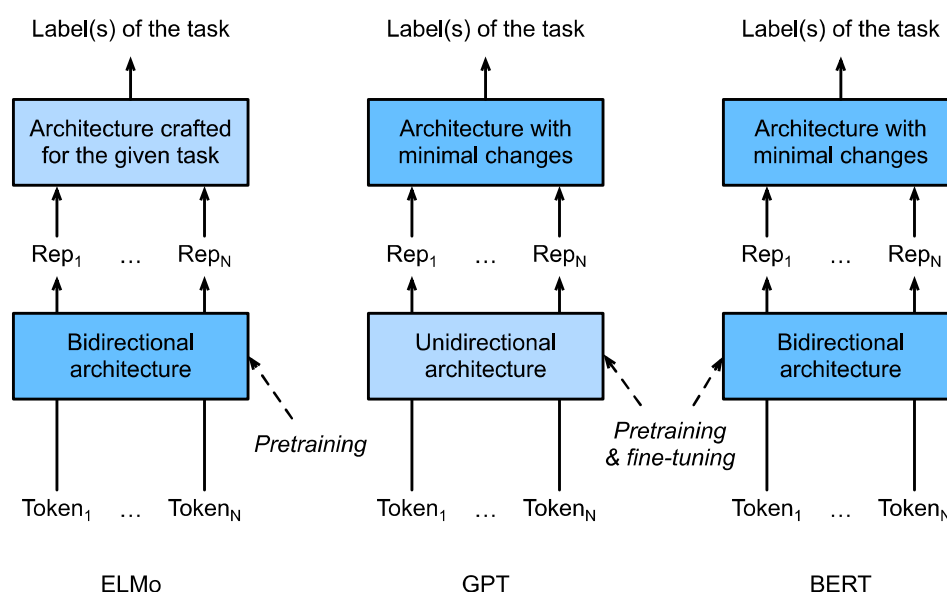


Fig. 14.7.1: A comparison of ELMo, GPT, and BERT.

BERT further improved the state of the art on eleven natural language processing tasks under broad categories of (i) single text classification (e.g., sentiment analysis), (ii) text pair classification (e.g., natural language inference), (iii) question answering, (iv) text tagging (e.g., named entity recognition). All proposed in 2018, from context-sensitive ELMo to task-agnostic GPT and BERT, conceptually simple yet empirically powerful pretraining of deep representations for natural languages have revolutionized solutions to various natural language processing tasks.

In the rest of this chapter, we will dive into the pretraining of BERT. When natural language processing applications are explained in Chapter 15, we will illustrate fine-tuning of BERT for downstream applications.

```
import torch
from torch import nn
from d2l import torch as d2l
```

### 14.7.4 Input Representation

In natural language processing, some tasks (e.g., sentiment analysis) take single text as the input, while in some other tasks (e.g., natural language inference), the input is a pair of text sequences. The BERT input sequence unambiguously represents both single text and text pairs. In the former, the BERT input sequence is the concatenation of the special classification token “<cls>”, tokens of a text sequence, and the special separation token “<sep>”. In the latter, the BERT input sequence is the concatenation of “<cls>”, tokens of the first text sequence, “<sep>”, tokens of the second text sequence, and “<sep>”. We will consistently distinguish the terminology “BERT input sequence” from other types of “sequences”. For instance, one *BERT input sequence* may include either one *text sequence* or two *text sequences*.

To distinguish text pairs, the learned segment embeddings  $\mathbf{e}_A$  and  $\mathbf{e}_B$  are added to the token embeddings of the first sequence and the second sequence, respectively. For single text inputs, only  $\mathbf{e}_A$  is used.

The following `get_tokens_and_segments` takes either one sentence or two sentences as the input, then returns tokens of the BERT input sequence and their corresponding segment IDs.

```
#@save
def get_tokens_and_segments(tokens_a, tokens_b=None):
    """Get tokens of the BERT input sequence and their segment IDs."""
    tokens = ['<cls>'] + tokens_a + ['<sep>']
    # 0 and 1 are marking segment A and B, respectively
    segments = [0] * (len(tokens_a) + 2)
    if tokens_b is not None:
        tokens += tokens_b + ['<sep>']
        segments += [1] * (len(tokens_b) + 1)
    return tokens, segments
```

BERT chooses the transformer encoder as its bidirectional architecture. Common in the transformer encoder, positional embeddings are added at every position of the BERT input sequence. However, different from the original transformer encoder, BERT uses *learnable* positional embeddings. To sum up, Fig. 14.7.2 shows that the embeddings of the BERT input sequence are the sum of the token embeddings, segment embeddings, and positional embeddings.

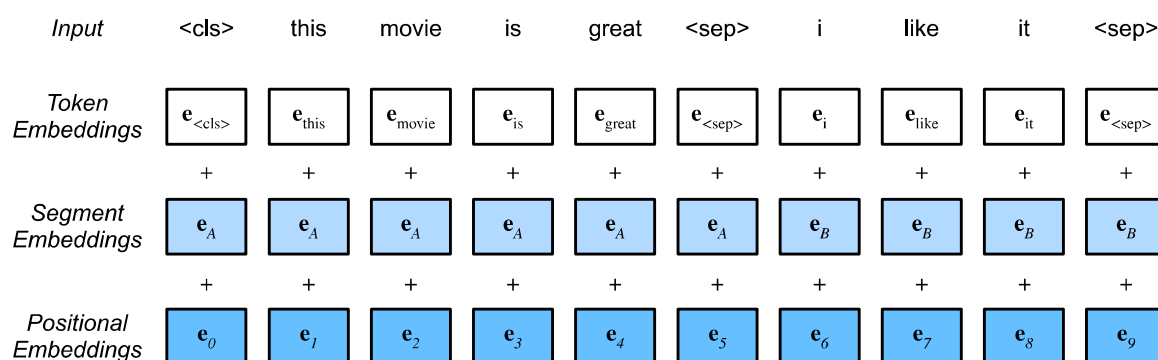


Fig. 14.7.2: The embeddings of the BERT input sequence are the sum of the token embeddings, segment embeddings, and positional embeddings.

The following `BERTEncoder` class is similar to the `TransformerEncoder` class as implemented in Section 10.7. Different from `TransformerEncoder`, `BERTEncoder` uses segment embeddings and learn-

able positional embeddings.

```
#@save
class BERTEncoder(nn.Module):
    """BERT encoder."""
    def __init__(self, vocab_size, num_hiddens, norm_shape, ffn_num_input,
                  ffn_num_hiddens, num_heads, num_layers, dropout,
                  max_len=1000, key_size=768, query_size=768, value_size=768,
                  **kwargs):
        super(BERTEncoder, self).__init__(**kwargs)
        self.token_embedding = nn.Embedding(vocab_size, num_hiddens)
        self.segment_embedding = nn.Embedding(2, num_hiddens)
        self.blks = nn.Sequential()
        for i in range(num_layers):
            self.blks.add_module(f"{i}", d2l.EncoderBlock(
                key_size, query_size, value_size, num_hiddens, norm_shape,
                ffn_num_input, ffn_num_hiddens, num_heads, dropout, True))
        # In BERT, positional embeddings are learnable, thus we create a
        # parameter of positional embeddings that are long enough
        self.pos_embedding = nn.Parameter(torch.randn(1, max_len,
                                                         num_hiddens))

    def forward(self, tokens, segments, valid_lens):
        # Shape of 'X' remains unchanged in the following code snippet:
        # (batch size, max sequence length, 'num_hiddens')
        X = self.token_embedding(tokens) + self.segment_embedding(segments)
        X = X + self.pos_embedding.data[:, :X.shape[1], :]
        for blk in self.blks:
            X = blk(X, valid_lens)
        return X
```

Suppose that the vocabulary size is 10000. To demonstrate forward inference of BERTEncoder, let us create an instance of it and initialize its parameters.

```
vocab_size, num_hiddens, ffn_num_hiddens, num_heads = 10000, 768, 1024, 4
norm_shape, ffn_num_input, num_layers, dropout = [768], 768, 2, 0.2
encoder = BERTEncoder(vocab_size, num_hiddens, norm_shape, ffn_num_input,
                      ffn_num_hiddens, num_heads, num_layers, dropout)
```

We define tokens to be 2 BERT input sequences of length 8, where each token is an index of the vocabulary. The forward inference of BERTEncoder with the input tokens returns the encoded result where each token is represented by a vector whose length is predefined by the hyperparameter `num_hiddens`. This hyperparameter is usually referred to as the *hidden size* (number of hidden units) of the transformer encoder.

```
tokens = torch.randint(0, vocab_size, (2, 8))
segments = torch.tensor([[0, 0, 0, 0, 1, 1, 1, 1], [0, 0, 0, 1, 1, 1, 1, 1]])
encoded_X = encoder(tokens, segments, None)
encoded_X.shape
```

```
torch.Size([2, 8, 768])
```

### 14.7.5 Pretraining Tasks

The forward inference of BERTEncoder gives the BERT representation of each token of the input text and the inserted special tokens “<cls>” and “<seq>”. Next, we will use these representations to compute the loss function for pretraining BERT. The pretraining is composed of the following two tasks: masked language modeling and next sentence prediction.

#### Masked Language Modeling

As illustrated in [Section 8.3](#), a language model predicts a token using the context on its left. To encode context bidirectionally for representing each token, BERT randomly masks tokens and uses tokens from the bidirectional context to predict the masked tokens in a self-supervised fashion. This task is referred to as a *masked language model*.

In this pretraining task, 15% of tokens will be selected at random as the masked tokens for prediction. To predict a masked token without cheating by using the label, one straightforward approach is to always replace it with a special “<mask>” token in the BERT input sequence. However, the artificial special token “<mask>” will never appear in fine-tuning. To avoid such a mismatch between pretraining and fine-tuning, if a token is masked for prediction (e.g., “great” is selected to be masked and predicted in “this movie is great”), in the input it will be replaced with:

- a special “<mask>” token for 80% of the time (e.g., “this movie is great” becomes “this movie is <mask>”);
- a random token for 10% of the time (e.g., “this movie is great” becomes “this movie is drink”);
- the unchanged label token for 10% of the time (e.g., “this movie is great” becomes “this movie is great”).

Note that for 10% of 15% time a random token is inserted. This occasional noise encourages BERT to be less biased towards the masked token (especially when the label token remains unchanged) in its bidirectional context encoding.

We implement the following MaskLM class to predict masked tokens in the masked language model task of BERT pretraining. The prediction uses a one-hidden-layer MLP (self.mlp). In forward inference, it takes two inputs: the encoded result of BERTEncoder and the token positions for prediction. The output is the prediction results at these positions.

```
#@save
class MaskLM(nn.Module):
    """The masked language model task of BERT."""
    def __init__(self, vocab_size, num_hiddens, num_inputs=768, **kwargs):
        super(MaskLM, self).__init__(**kwargs)
        self.mlp = nn.Sequential(nn.Linear(num_inputs, num_hiddens),
                                nn.ReLU(),
                                nn.LayerNorm(num_hiddens),
                                nn.Linear(num_hiddens, vocab_size))

    def forward(self, X, pred_positions):
        num_pred_positions = pred_positions.shape[1]
        pred_positions = pred_positions.reshape(-1)
        batch_size = X.shape[0]
        batch_idx = torch.arange(0, batch_size)
        # Suppose that `batch_size` = 2, `num_pred_positions` = 3, then
```

(continues on next page)

(continued from previous page)

```
# `batch_idx` is `torch.tensor([0, 0, 0, 1, 1, 1])`
batch_idx = torch.repeat_interleave(batch_idx, num_pred_positions)
masked_X = X[batch_idx, pred_positions]
masked_X = masked_X.reshape((batch_size, num_pred_positions, -1))
mlm_Y_hat = self.mlp(masked_X)
return mlm_Y_hat
```

To demonstrate the forward inference of MaskLM, we create its instance `mlm` and initialize it. Recall that `encoded_X` from the forward inference of BERTEncoder represents 2 BERT input sequences. We define `mlm_positions` as the 3 indices to predict in either BERT input sequence of `encoded_X`. The forward inference of `mlm` returns prediction results `mlm_Y_hat` at all the masked positions `mlm_positions` of `encoded_X`. For each prediction, the size of the result is equal to the vocabulary size.

```
mlm = MaskLM(vocab_size, num_hiddens)
mlm_positions = torch.tensor([[1, 5, 2], [6, 1, 5]])
mlm_Y_hat = mlm(encoded_X, mlm_positions)
mlm_Y_hat.shape
```

```
torch.Size([2, 3, 10000])
```

With the ground truth labels `mlm_Y` of the predicted tokens `mlm_Y_hat` under masks, we can calculate the cross-entropy loss of the masked language model task in BERT pretraining.

```
mlm_Y = torch.tensor([[7, 8, 9], [10, 20, 30]])
loss = nn.CrossEntropyLoss(reduction='none')
mlm_l = loss(mlm_Y_hat.reshape((-1, vocab_size)), mlm_Y.reshape(-1))
mlm_l.shape
```

```
torch.Size([6])
```

## Next Sentence Prediction

Although masked language modeling is able to encode bidirectional context for representing words, it does not explicitly model the logical relationship between text pairs. To help understand the relationship between two text sequences, BERT considers a binary classification task, *next sentence prediction*, in its pretraining. When generating sentence pairs for pretraining, for half of the time they are indeed consecutive sentences with the label “True”; while for the other half of the time the second sentence is randomly sampled from the corpus with the label “False”.

The following `NextSentencePred` class uses a one-hidden-layer MLP to predict whether the second sentence is the next sentence of the first in the BERT input sequence. Due to self-attention in the transformer encoder, the BERT representation of the special token “<cls>” encodes both the two sentences from the input. Hence, the output layer (`self.output`) of the MLP classifier takes `X` as the input, where `X` is the output of the MLP hidden layer whose input is the encoded “<cls>” token.

```
@save
class NextSentencePred(nn.Module):
```

(continues on next page)



(continued from previous page)

```
"""The next sentence prediction task of BERT."""
def __init__(self, num_inputs, **kwargs):
    super(NextSentencePred, self).__init__(**kwargs)
    self.output = nn.Linear(num_inputs, 2)

def forward(self, X):
    # `X` shape: (batch size, `num_hiddens`)
    return self.output(X)
```

We can see that the forward inference of an `NextSentencePred` instance returns binary predictions for each BERT input sequence.

```
# PyTorch by default won't flatten the tensor as seen in mxnet where, if
# flatten=True, all but the first axis of input data are collapsed together
encoded_X = torch.flatten(encoded_X, start_dim=1)
# input_shape for NSP: (batch size, `num_hiddens`)
nsp = NextSentencePred(encoded_X.shape[-1])
nsp_Y_hat = nsp(encoded_X)
nsp_Y_hat.shape
```

```
torch.Size([2, 2])
```

The cross-entropy loss of the 2 binary classifications can also be computed.

```
nsp_y = torch.tensor([0, 1])
nsp_l = loss(nsp_Y_hat, nsp_y)
nsp_l.shape
```

```
torch.Size([2])
```

It is noteworthy that all the labels in both the aforementioned pretraining tasks can be trivially obtained from the pretraining corpus without manual labeling effort. The original BERT has been pretrained on the concatenation of BookCorpus (Zhu et al., 2015) and English Wikipedia. These two text corpora are huge: they have 800 million words and 2.5 billion words, respectively.

### 14.7.6 Putting All Things Together

When pretraining BERT, the final loss function is a linear combination of both the loss functions for masked language modeling and next sentence prediction. Now we can define the `BERTModel` class by instantiating the three classes `BERTEncoder`, `MaskLM`, and `NextSentencePred`. The forward inference returns the encoded BERT representations `encoded_X`, predictions of masked language modeling `mlm_Y_hat`, and next sentence predictions `nsp_Y_hat`.

```
@save
class BERTModel(nn.Module):
    """The BERT model."""
    def __init__(self, vocab_size, num_hiddens, norm_shape, ffn_num_input,
                  ffn_num_hiddens, num_heads, num_layers, dropout,
                  max_len=1000, key_size=768, query_size=768, value_size=768,
```

(continues on next page)



```

        hid_in_features=768, mlm_in_features=768,
        nsp_in_features=768):
    super(BERTModel, self).__init__()
    self.encoder = BERTEncoder(vocab_size, num_hiddens, norm_shape,
                               ffn_num_input, ffn_num_hiddens, num_heads, num_layers,
                               dropout, max_len=max_len, key_size=key_size,
                               query_size=query_size, value_size=value_size)
    self.hidden = nn.Sequential(nn.Linear(hid_in_features, num_hiddens),
                                nn.Tanh())
    self.mlm = MaskLM(vocab_size, num_hiddens, mlm_in_features)
    self.nsp = NextSentencePred(nsp_in_features)

    def forward(self, tokens, segments, valid_lens=None, pred_positions=None):
        encoded_X = self.encoder(tokens, segments, valid_lens)
        if pred_positions is not None:
            mlm_Y_hat = self.mlm(encoded_X, pred_positions)
        else:
            mlm_Y_hat = None
        # The hidden layer of the MLP classifier for next sentence prediction.
        # 0 is the index of the '<cls>' token
        nsp_Y_hat = self.nsp(self.hidden(encoded_X[:, 0, :]))
        return encoded_X, mlm_Y_hat, nsp_Y_hat

```

## Summary

- Word embedding models such as word2vec and GloVe are context-independent. They assign the same pretrained vector to the same word regardless of the context of the word (if any). It is hard for them to handle well polysemy or complex semantics in natural languages.
- For context-sensitive word representations such as ELMo and GPT, representations of words depend on their contexts.
- ELMo encodes context bidirectionally but uses task-specific architectures (however, it is practically non-trivial to craft a specific architecture for every natural language processing task); while GPT is task-agnostic but encodes context left-to-right.
- BERT combines the best of both worlds: it encodes context bidirectionally and requires minimal architecture changes for a wide range of natural language processing tasks.
- The embeddings of the BERT input sequence are the sum of the token embeddings, segment embeddings, and positional embeddings.
- Pretraining BERT is composed of two tasks: masked language modeling and next sentence prediction. The former is able to encode bidirectional context for representing words, while the latter explicitly models the logical relationship between text pairs.

## Exercises

1. Why does BERT succeed?
2. All other things being equal, will a masked language model require more or fewer pretraining steps to converge than a left-to-right language model? Why?
3. In the original implementation of BERT, the positionwise feed-forward network in BERTEncoder (via `d2l.EncoderBlock`) and the fully-connected layer in MaskLM both use the Gaussian error linear unit (GELU) (Hendrycks & Gimpel, 2016) as the activation function. Research into the difference between GELU and ReLU.

## 14.8 The Dataset for Pretraining BERT

To pretrain the BERT model as implemented in [Section 14.7](#), we need to generate the dataset in the ideal format to facilitate the two pretraining tasks: masked language modeling and next sentence prediction. On one hand, the original BERT model is pretrained on the concatenation of two huge corpora BookCorpus and English Wikipedia (see [Section 14.7.5](#)), making it hard to run for most readers of this book. On the other hand, the off-the-shelf pretrained BERT model may not fit for applications from specific domains like medicine. Thus, it is getting popular to pretrain BERT on a customized dataset. To facilitate the demonstration of BERT pretraining, we use a smaller corpus WikiText-2 (Merity et al., 2016).

Comparing with the PTB dataset used for pretraining word2vec in [Section 14.3](#), WikiText-2 (i) retains the original punctuation, making it suitable for next sentence prediction; (ii) retains the original case and numbers; (iii) is over twice larger.

```
import os
import random
import torch
from d2l import torch as d2l
```

In the WikiText-2 dataset, each line represents a paragraph where space is inserted between any punctuation and its preceding token. Paragraphs with at least two sentences are retained. To split sentences, we only use the period as the delimiter for simplicity. We leave discussions of more complex sentence splitting techniques in the exercises at the end of this section.

```
#@save
d2l.DATA_HUB['wikitext-2'] = (
    'https://s3.amazonaws.com/research.metamind.io/wikitext/'
    'wikitext-2-v1.zip', '3c914d17d80b1459be871a5039ac23e752a53cbe')

#@save
def _read_wiki(data_dir):
    file_name = os.path.join(data_dir, 'wiki.train.tokens')
    with open(file_name, 'r') as f:
        lines = f.readlines()
    # Uppercase letters are converted to lowercase ones
    paragraphs = [line.strip().lower().split(' . ')]
```

(continues on next page)

```

        for line in lines if len(line.split(' . ')) >= 2]
    random.shuffle(paragraphs)
    return paragraphs

```

### 14.8.1 Defining Helper Functions for Pretraining Tasks

In the following, we begin by implementing helper functions for the two BERT pretraining tasks: next sentence prediction and masked language modeling. These helper functions will be invoked later when transforming the raw text corpus into the dataset of the ideal format to pretrain BERT.

#### Generating the Next Sentence Prediction Task

According to descriptions of [Section 14.7.5](#), the `_get_next_sentence` function generates a training example for the binary classification task.

```

#@save
def _get_next_sentence(sentence, next_sentence, paragraphs):
    if random.random() < 0.5:
        is_next = True
    else:
        # `paragraphs` is a list of lists of lists
        next_sentence = random.choice(random.choice(paragraphs))
        is_next = False
    return sentence, next_sentence, is_next

```

The following function generates training examples for next sentence prediction from the input paragraph by invoking the `_get_next_sentence` function. Here paragraph is a list of sentences, where each sentence is a list of tokens. The argument `max_len` specifies the maximum length of a BERT input sequence during pretraining.

```

#@save
def _get_nsp_data_from_paragraph(paragraph, paragraphs, vocab, max_len):
    nsp_data_from_paragraph = []
    for i in range(len(paragraph) - 1):
        tokens_a, tokens_b, is_next = _get_next_sentence(
            paragraph[i], paragraph[i + 1], paragraphs)
        # Consider 1 '<cls>' token and 2 '<sep>' tokens
        if len(tokens_a) + len(tokens_b) + 3 > max_len:
            continue
        tokens, segments = d2l.get_tokens_and_segments(tokens_a, tokens_b)
        nsp_data_from_paragraph.append((tokens, segments, is_next))
    return nsp_data_from_paragraph

```

## Generating the Masked Language Modeling Task

In order to generate training examples for the masked language modeling task from a BERT input sequence, we define the following `_replace_mlm_tokens` function. In its inputs, `tokens` is a list of tokens representing a BERT input sequence, `candidate_pred_positions` is a list of token indices of the BERT input sequence excluding those of special tokens (special tokens are not predicted in the masked language modeling task), and `num_mlm_preds` indicates the number of predictions (recall 15% random tokens to predict). Following the definition of the masked language modeling task in [Section 14.7.5](#), at each prediction position, the input may be replaced by a special “<mask>” token or a random token, or remain unchanged. In the end, the function returns the input tokens after possible replacement, the token indices where predictions take place and labels for these predictions.

```
@save
def _replace_mlm_tokens(tokens, candidate_pred_positions, num_mlm_preds,
                        vocab):
    # Make a new copy of tokens for the input of a masked language model,
    # where the input may contain replaced '<mask>' or random tokens
    mlm_input_tokens = [token for token in tokens]
    pred_positions_and_labels = []
    # Shuffle for getting 15% random tokens for prediction in the masked
    # language modeling task
    random.shuffle(candidate_pred_positions)
    for mlm_pred_position in candidate_pred_positions:
        if len(pred_positions_and_labels) >= num_mlm_preds:
            break
        masked_token = None
        # 80% of the time: replace the word with the '<mask>' token
        if random.random() < 0.8:
            masked_token = '<mask>'
        else:
            # 10% of the time: keep the word unchanged
            if random.random() < 0.5:
                masked_token = tokens[mlm_pred_position]
            # 10% of the time: replace the word with a random word
            else:
                masked_token = random.choice(vocab.idx_to_token)
        mlm_input_tokens[mlm_pred_position] = masked_token
        pred_positions_and_labels.append(
            (mlm_pred_position, tokens[mlm_pred_position]))
    return mlm_input_tokens, pred_positions_and_labels
```

By invoking the aforementioned `_replace_mlm_tokens` function, the following function takes a BERT input sequence (`tokens`) as an input and returns indices of the input tokens (after possible token replacement as described in [Section 14.7.5](#)), the token indices where predictions take place, and label indices for these predictions.

```
@save
def _get_mlm_data_from_tokens(tokens, vocab):
    candidate_pred_positions = []
    # 'tokens' is a list of strings
    for i, token in enumerate(tokens):
        # Special tokens are not predicted in the masked language modeling
        # task
```

(continues on next page)

```

    if token in ['<cls>', '<sep>']:
        continue
    candidate_pred_positions.append(i)
    # 15% of random tokens are predicted in the masked language modeling task
    num_mlm_preds = max(1, round(len(tokens) * 0.15))
    mlm_input_tokens, pred_positions_and_labels = _replace_mlm_tokens(
        tokens, candidate_pred_positions, num_mlm_preds, vocab)
    pred_positions_and_labels = sorted(pred_positions_and_labels,
                                       key=lambda x: x[0])
    pred_positions = [v[0] for v in pred_positions_and_labels]
    mlm_pred_labels = [v[1] for v in pred_positions_and_labels]
    return vocab[mlm_input_tokens], pred_positions, vocab[mlm_pred_labels]

```

### 14.8.2 Transforming Text into the Pretraining Dataset

Now we are almost ready to customize a Dataset class for pretraining BERT. Before that, we still need to define a helper function `_pad_bert_inputs` to append the special “<mask>” tokens to the inputs. Its argument examples contain the outputs from the helper functions `_get_nsp_data_from_paragraph` and `_get_mlm_data_from_tokens` for the two pretraining tasks.

```

#@save
def _pad_bert_inputs(examples, max_len, vocab):
    max_num_mlm_preds = round(max_len * 0.15)
    all_token_ids, all_segments, valid_lens, = [], [], []
    all_pred_positions, all_mlm_weights, all_mlm_labels = [], [], []
    nsp_labels = []
    for (token_ids, pred_positions, mlm_pred_label_ids, segments,
         is_next) in examples:
        all_token_ids.append(torch.tensor(token_ids + [vocab['<pad>']] * (
            max_len - len(token_ids)), dtype=torch.long))
        all_segments.append(torch.tensor(segments + [0] * (
            max_len - len(segments)), dtype=torch.long))
        # 'valid_lens' excludes count of '<pad>' tokens
        valid_lens.append(torch.tensor(len(token_ids), dtype=torch.float32))
        all_pred_positions.append(torch.tensor(pred_positions + [0] * (
            max_num_mlm_preds - len(pred_positions)), dtype=torch.long))
        # Predictions of padded tokens will be filtered out in the loss via
        # multiplication of 0 weights
        all_mlm_weights.append(
            torch.tensor([1.0] * len(mlm_pred_label_ids) + [0.0] * (
                max_num_mlm_preds - len(pred_positions)),
                dtype=torch.float32))
        all_mlm_labels.append(torch.tensor(mlm_pred_label_ids + [0] * (
            max_num_mlm_preds - len(mlm_pred_label_ids)), dtype=torch.long))
        nsp_labels.append(torch.tensor(is_next, dtype=torch.long))
    return (all_token_ids, all_segments, valid_lens, all_pred_positions,
            all_mlm_weights, all_mlm_labels, nsp_labels)

```

Putting the helper functions for generating training examples of the two pretraining tasks, and the helper function for padding inputs together, we customize the following `WikiTextDataset` class as the WikiText-2 dataset for pretraining BERT. By implementing the `__getitem__` function, we can arbitrarily access the pretraining (masked language modeling and next sentence prediction) examples generated from a pair of sentences from the WikiText-2 corpus.

The original BERT model uses WordPiece embeddings whose vocabulary size is 30000 (Wu et al., 2016). The tokenization method of WordPiece is a slight modification of the original byte pair encoding algorithm in `subsec_Byte_Pair_Encoding`. For simplicity, we use the `d2l.tokenize` function for tokenization. Infrequent tokens that appear less than five times are filtered out.

```
#@save
class _WikiTextDataset(torch.utils.data.Dataset):
    def __init__(self, paragraphs, max_len):
        # Input 'paragraphs[i]' is a list of sentence strings representing a
        # paragraph; while output 'paragraphs[i]' is a list of sentences
        # representing a paragraph, where each sentence is a list of tokens
        paragraphs = [d2l.tokenize(
            paragraph, token='word') for paragraph in paragraphs]
        sentences = [sentence for paragraph in paragraphs
                     for sentence in paragraph]
        self.vocab = d2l.Vocab(sentences, min_freq=5, reserved_tokens=[
            '<pad>', '<mask>', '<cls>', '<sep>'])
        # Get data for the next sentence prediction task
        examples = []
        for paragraph in paragraphs:
            examples.extend(_get_nsp_data_from_paragraph(
                paragraph, paragraphs, self.vocab, max_len))
        # Get data for the masked language model task
        examples = [(_get_mlm_data_from_tokens(tokens, self.vocab)
                     + (segments, is_next))
                    for tokens, segments, is_next in examples]
        # Pad inputs
        (self.all_token_ids, self.all_segments, self.valid_lens,
         self.all_pred_positions, self.all_mlm_weights,
         self.all_mlm_labels, self.nsp_labels) = _pad_bert_inputs(
            examples, max_len, self.vocab)

    def __getitem__(self, idx):
        return (self.all_token_ids[idx], self.all_segments[idx],
                self.valid_lens[idx], self.all_pred_positions[idx],
                self.all_mlm_weights[idx], self.all_mlm_labels[idx],
                self.nsp_labels[idx])

    def __len__(self):
        return len(self.all_token_ids)
```

By using the `_read_wiki` function and the `_WikiTextDataset` class, we define the following `load_data_wiki` to download and WikiText-2 dataset and generate pretraining examples from it.

```
#@save
def load_data_wiki(batch_size, max_len):
    """Load the WikiText-2 dataset."""
    num_workers = d2l.get_dataloader_workers()
    data_dir = d2l.download_extract('wikitext-2', 'wikitext-2')
    paragraphs = _read_wiki(data_dir)
    train_set = _WikiTextDataset(paragraphs, max_len)
    train_iter = torch.utils.data.DataLoader(train_set, batch_size,
                                              shuffle=True, num_workers=num_workers)
    return train_iter, train_set.vocab
```

Setting the batch size to 512 and the maximum length of a BERT input sequence to be 64, we

print out the shapes of a minibatch of BERT pretraining examples. Note that in each BERT input sequence, 10 ( $64 \times 0.15$ ) positions are predicted for the masked language modeling task.

```
batch_size, max_len = 512, 64
train_iter, vocab = load_data_wiki(batch_size, max_len)

for (tokens_X, segments_X, valid_lens_x, pred_positions_X, mlm_weights_X,
     mlm_Y, nsp_y) in train_iter:
    print(tokens_X.shape, segments_X.shape, valid_lens_x.shape,
          pred_positions_X.shape, mlm_weights_X.shape, mlm_Y.shape,
          nsp_y.shape)
    break
```

```
torch.Size([512, 64]) torch.Size([512, 64]) torch.Size([512]) torch.Size([512, 10]) torch.
↪Size([512, 10]) torch.Size([512, 10]) torch.Size([512])
```

In the end, let us take a look at the vocabulary size. Even after filtering out infrequent tokens, it is still over twice larger than that of the PTB dataset.

```
len(vocab)
```

```
20256
```

## Summary

- Comparing with the PTB dataset, the WikiText-2 dataset retains the original punctuation, case and numbers, and is over twice larger.
- We can arbitrarily access the pretraining (masked language modeling and next sentence prediction) examples generated from a pair of sentences from the WikiText-2 corpus.

## Exercises

1. For simplicity, the period is used as the only delimiter for splitting sentences. Try other sentence splitting techniques, such as the spaCy and NLTK. Take NLTK as an example. You need to install NLTK first: `pip install nltk`. In the code, first import `nltk`. Then, download the Punkt sentence tokenizer: `nltk.download('punkt')`. To split sentences such as `sentences = 'This is great ! Why not ?'`, invoking `nltk.tokenize.sent_tokenize(sentences)` will return a list of two sentence strings: `['This is great !', 'Why not ?']`.
2. What is the vocabulary size if we do not filter out any infrequent token?



## 14.9 Pretraining BERT

With the BERT model implemented in [Section 14.7](#) and the pretraining examples generated from the WikiText-2 dataset in [Section 14.8](#), we will pretrain BERT on the WikiText-2 dataset in this section.

```
import torch
from torch import nn
from d2l import torch as d2l
```

To start, we load the WikiText-2 dataset as minibatches of pretraining examples for masked language modeling and next sentence prediction. The batch size is 512 and the maximum length of a BERT input sequence is 64. Note that in the original BERT model, the maximum length is 512.

```
batch_size, max_len = 512, 64
train_iter, vocab = d2l.load_data_wiki(batch_size, max_len)
```

```
Downloading ../data/wikitext-2-v1.zip from https://s3.amazonaws.com/research.metamind.io/
wikitext/wikitext-2-v1.zip...
```

### 14.9.1 Pretraining BERT

The original BERT has two versions of different model sizes ([Devlin et al., 2018](#)). The base model (BERT<sub>BASE</sub>) uses 12 layers (transformer encoder blocks) with 768 hidden units (hidden size) and 12 self-attention heads. The large model (BERT<sub>LARGE</sub>) uses 24 layers with 1024 hidden units and 16 self-attention heads. Notably, the former has 110 million parameters while the latter has 340 million parameters. For demonstration with ease, we define a small BERT, using 2 layers, 128 hidden units, and 2 self-attention heads.

```
net = d2l.BERTModel(len(vocab), num_hiddens=128, norm_shape=[128],
                    ffn_num_hiddens=256, num_heads=2,
                    num_layers=2, dropout=0.2, key_size=128, query_size=128,
                    value_size=128, hid_in_features=128, mlm_in_features=128,
                    nsp_in_features=128)
devices = d2l.try_all_gpus()
loss = nn.CrossEntropyLoss()
```

Before defining the training loop, we define a helper function `_get_batch_loss_bert`. Given the shard of training examples, this function computes the loss for both the masked language modeling and next sentence prediction tasks. Note that the final loss of BERT pretraining is just the sum of both the masked language modeling loss and the next sentence prediction loss.

```
# @save
def _get_batch_loss_bert(net, loss, vocab_size, tokens_X,
                        segments_X, valid_lens_x,
                        pred_positions_X, mlm_weights_X,
                        mlm_Y, nsp_Y):
    # Forward pass
    _, mlm_Y_hat, nsp_Y_hat = net(tokens_X, segments_X,
                                valid_lens_x.reshape(-1),
```

(continues on next page)

```

                                pred_positions_X)
# Compute masked language model loss
mlm_l = loss(mlm_Y_hat.reshape(-1, vocab_size), mlm_Y.reshape(-1)) *\
mlm_weights_X.reshape(-1, 1)
mlm_l = mlm_l.sum() / (mlm_weights_X.sum() + 1e-8)
# Compute next sentence prediction loss
nsp_l = loss(nsp_Y_hat, nsp_y)
l = mlm_l + nsp_l
return mlm_l, nsp_l, l

```

Invoking the two aforementioned helper functions, the following `train_bert` function defines the procedure to pretrain BERT (net) on the WikiText-2 (`train_iter`) dataset. Training BERT can take very long. Instead of specifying the number of epochs for training as in the `train_ch13` function (see [Section 13.1](#)), the input `num_steps` of the following function specifies the number of iteration steps for training.

```

def train_bert(train_iter, net, loss, vocab_size, devices, num_steps):
    net = nn.DataParallel(net, device_ids=devices).to(devices[0])
    trainer = torch.optim.Adam(net.parameters(), lr=0.01)
    step, timer = 0, d2l.Timer()
    animator = d2l.Animator(xlabel='step', ylabel='loss',
                            xlim=[1, num_steps], legend=['mlm', 'nsp'])
    # Sum of masked language modeling losses, sum of next sentence prediction
    # losses, no. of sentence pairs, count
    metric = d2l.Accumulator(4)
    num_steps_reached = False
    while step < num_steps and not num_steps_reached:
        for tokens_X, segments_X, valid_lens_x, pred_positions_X, \
            mlm_weights_X, mlm_Y, nsp_y in train_iter:
            tokens_X = tokens_X.to(devices[0])
            segments_X = segments_X.to(devices[0])
            valid_lens_x = valid_lens_x.to(devices[0])
            pred_positions_X = pred_positions_X.to(devices[0])
            mlm_weights_X = mlm_weights_X.to(devices[0])
            mlm_Y, nsp_y = mlm_Y.to(devices[0]), nsp_y.to(devices[0])
            trainer.zero_grad()
            timer.start()
            mlm_l, nsp_l, l = _get_batch_loss_bert(
                net, loss, vocab_size, tokens_X, segments_X, valid_lens_x,
                pred_positions_X, mlm_weights_X, mlm_Y, nsp_y)
            l.backward()
            trainer.step()
            metric.add(mlm_l, nsp_l, tokens_X.shape[0], 1)
            timer.stop()
            animator.add(step + 1,
                        (metric[0] / metric[3], metric[1] / metric[3]))
            step += 1
        if step == num_steps:
            num_steps_reached = True
            break

    print(f'MLM loss {metric[0] / metric[3]:.3f}, '
          f'NSP loss {metric[1] / metric[3]:.3f}')
    print(f'{metric[2] / timer.sum():.1f} sentence pairs/sec on '

```

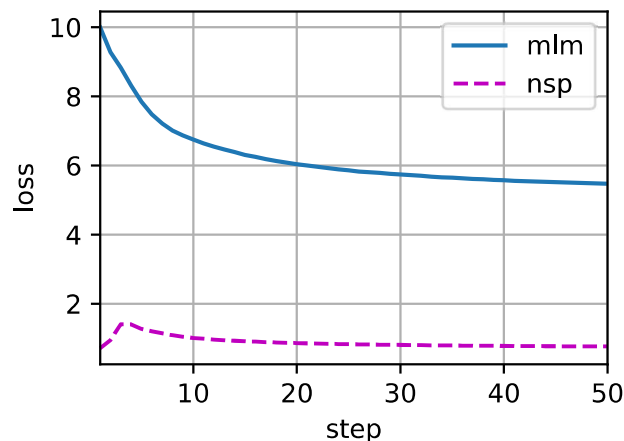
(continues on next page)

```
f'{str(devices)}')
```

We can plot both the masked language modeling loss and the next sentence prediction loss during BERT pretraining.

```
train_bert(train_iter, net, loss, len(vocab), devices, 50)
```

```
MLM loss 5.468, NSP loss 0.763
4614.3 sentence pairs/sec on [device(type='cuda', index=0), device(type='cuda', index=1)]
```



### 14.9.2 Representing Text with BERT

After pretraining BERT, we can use it to represent single text, text pairs, or any token in them. The following function returns the BERT (net) representations for all tokens in `tokens_a` and `tokens_b`.

```
def get_bert_encoding(net, tokens_a, tokens_b=None):
    tokens, segments = d2l.get_tokens_and_segments(tokens_a, tokens_b)
    token_ids = torch.tensor(vocab[tokens], device=devices[0]).unsqueeze(0)
    segments = torch.tensor(segments, device=devices[0]).unsqueeze(0)
    valid_len = torch.tensor(len(tokens), device=devices[0]).unsqueeze(0)
    encoded_X, _, _ = net(token_ids, segments, valid_len)
    return encoded_X
```

Consider the sentence “a crane is flying”. Recall the input representation of BERT as discussed in [Section 14.7.4](#). After inserting special tokens “<cls>” (used for classification) and “<sep>” (used for separation), the BERT input sequence has a length of six. Since zero is the index of the “<cls>” token, `encoded_text[:, 0, :]` is the BERT representation of the entire input sentence. To evaluate the polysemy token “crane”, we also print out the first three elements of the BERT representation of the token.

```
tokens_a = ['a', 'crane', 'is', 'flying']
encoded_text = get_bert_encoding(net, tokens_a)
# Tokens: '<cls>', 'a', 'crane', 'is', 'flying', '<sep>'
encoded_text_cls = encoded_text[:, 0, :]
```

(continues on next page)

```
encoded_text_crane = encoded_text[:, 2, :]
encoded_text.shape, encoded_text_cls.shape, encoded_text_crane[0][:3]
```

```
(torch.Size([1, 6, 128]),
 torch.Size([1, 128]),
 tensor([-1.7630, -0.1749,  0.6695], device='cuda:0', grad_fn=<SliceBackward0>))
```

Now consider a sentence pair “a crane driver came” and “he just left”. Similarly, `encoded_pair[:, 0, :]` is the encoded result of the entire sentence pair from the pretrained BERT. Note that the first three elements of the polysemy token “crane” are different from those when the context is different. This supports that BERT representations are context-sensitive.

```
tokens_a, tokens_b = ['a', 'crane', 'driver', 'came'], ['he', 'just', 'left']
encoded_pair = get_bert_encoding(net, tokens_a, tokens_b)
# Tokens: '<cls>', 'a', 'crane', 'driver', 'came', '<sep>', 'he', 'just',
# 'left', '<sep>'
encoded_pair_cls = encoded_pair[:, 0, :]
encoded_pair_crane = encoded_pair[:, 2, :]
encoded_pair.shape, encoded_pair_cls.shape, encoded_pair_crane[0][:3]
```

```
(torch.Size([1, 10, 128]),
 torch.Size([1, 128]),
 tensor([-0.1681, -0.9842,  0.3190], device='cuda:0', grad_fn=<SliceBackward0>))
```

In [Chapter 15](#), we will fine-tune a pretrained BERT model for downstream natural language processing applications.

## Summary

- The original BERT has two versions, where the base model has 110 million parameters and the large model has 340 million parameters.
- After pretraining BERT, we can use it to represent single text, text pairs, or any token in them.
- In the experiment, the same token has different BERT representation when their contexts are different. This supports that BERT representations are context-sensitive.

## Exercises

1. In the experiment, we can see that the masked language modeling loss is significantly higher than the next sentence prediction loss. Why?
2. Set the maximum length of a BERT input sequence to be 512 (same as the original BERT model). Use the configurations of the original BERT model such as `BERT_LARGE`. Do you encounter any error when running this section? Why?